

Multi-variable chain rule for differentiation

$$z = f(x(t), y(t))$$

$$\frac{\partial z}{\partial t} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial t} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial t}$$

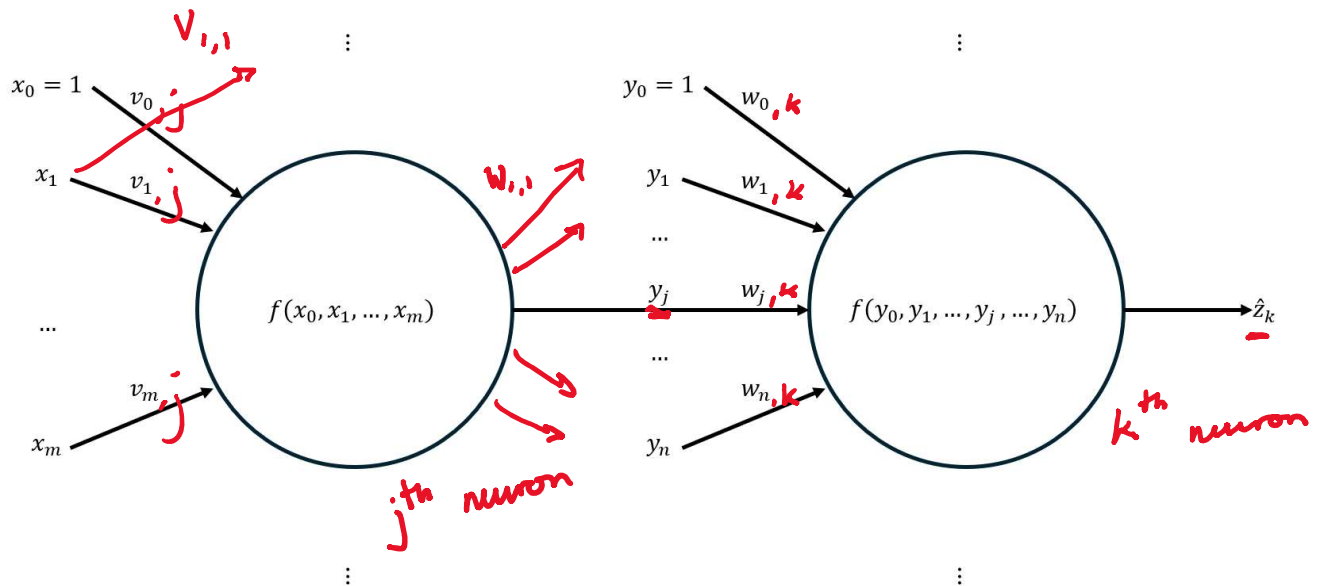
Example $f(x, y) = x^2 + y^2$
 $x(t) = \ln(t)$
 $y(t) = \sin(t)$

$$\frac{\partial z}{\partial t} = 2x \cdot \frac{1}{t} + 2y \cdot \cos(t)$$

same

$$z = \ln(t)^2 + \sin(t)^2$$

$$\frac{\partial z}{\partial t} = 2 \ln(t) \cdot \frac{1}{t} + 2 \sin(t) \cdot \cos(t)$$



We want $\frac{\partial L}{\partial v_i}$ for all i .

$$r_j = \sum_l v_l x_l \quad \text{aggregation}$$

$$y_j = a(r_j) \quad \text{activation}$$

We want

$$\frac{\partial L}{\partial v_i} = \frac{\partial L}{\partial r_j} \frac{\partial r_j}{\partial v_i}$$

$$r_j = \sum_l v_l x_l$$

$$\frac{\partial r_j}{\partial v_i} = x_i$$

$$\frac{\partial L}{\partial r_j} = \frac{\partial L}{\partial y_j} \frac{\partial y_j}{\partial r_j}$$

$$a'(r_j)$$

$$\frac{\partial L}{\partial r_j} = \frac{\partial y_j}{\partial r_j} = a'(r_j)$$

$$L = f(s_1(y_j), s_2(y_j), s_3(y_j), \dots)$$

$$\frac{\partial L}{\partial y_j} = \frac{\partial L}{\partial s_1} \frac{\partial s_1}{\partial y_j} + \frac{\partial L}{\partial s_2} \frac{\partial s_2}{\partial y_j} + \frac{\partial L}{\partial s_3} \frac{\partial s_3}{\partial y_j} + \dots$$

$$\frac{\partial L}{\partial y_j} = \sum_k \frac{\partial L}{\partial s_k} \frac{\partial s_k}{\partial y_j}$$

$$s_k = \sum_l w_{kl} y_l$$

$$\frac{\partial s_k}{\partial y_j} = w_{kj}$$

$$\begin{aligned} \frac{\partial L}{\partial v_i} &= \frac{\partial L}{\partial r_j} \frac{\partial r_j}{\partial v_i} \\ &= \left[\sum_k \frac{\partial L}{\partial s_k} w_{kj} a'(r_j) \right] x_i \end{aligned}$$

$\frac{\partial L}{\partial s_k}$ was already computed when we computed change in loss with respect to

computed change in loss
weights in the last layer.

We backpropagate the derivative through
the network.

This is independent of whether subsequent layers
is output or non-output layer.

Backpropagation

Given a loss function L , we want to compute
 $\frac{\partial L}{\partial w_i}$ where i indexes all weights in the network.

0. Forward propagate input through network.

Store aggregations + activations in memory

For each layer from output to input layer:

Define s_k to be the aggregation in k^{th}
neuron of subsequent layer.

Define $w_{j,k}$ to be the weight of the connection
associate with j^{th} input and k^{th} neuron of
subsequent layer

Define r_j as the aggregation in j^{th} neuron

Define r_j as the aggregation in j of current layer.

Define $v_{i,j}$ as the weight of the connection associated with i^{th} input and j^{th} neuron of current layer.

Case 1

Current layer is output layer

1. $\frac{\partial L}{\partial r_j} = \frac{\partial L}{\partial \hat{z}_j} a'(r_j)$. Store in memory.
for all nodes j .

2. $\frac{\partial L}{\partial v_{i,j}} = \frac{\partial L}{\partial r_j} x_i$.

Case 2

Current layer is not output layer.

1. Retrieve $\frac{\partial L}{\partial s_k}$ from memory for all nodes k .

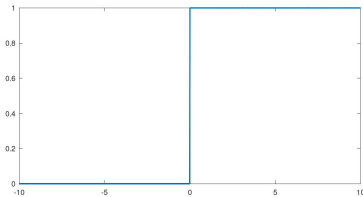
2. $\frac{\partial L}{\partial r_j} = \sum_k \frac{\partial L}{\partial s_k} w_{j,k} \underline{a'(r_j)}$
for all nodes j . Store in memory.

for all nodes j .

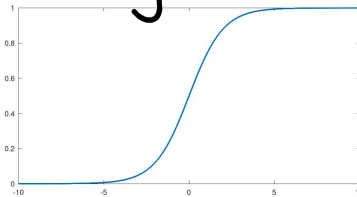
$$3. \frac{\partial L}{\partial v_{i,j}} = \frac{\partial L}{\partial r_j} x_i$$

Differentiating Activation Functions

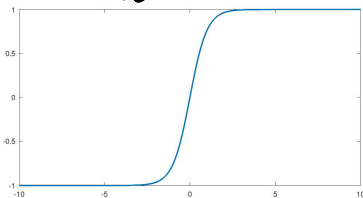
Heaviside



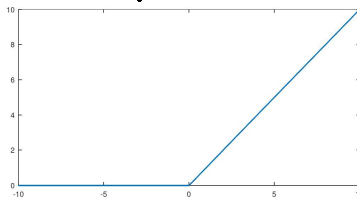
Logistic



Tanh



ReLU



Heaviside function:

$$a(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$a'(x) = 0$$

Logistic:

$$a(x) = \frac{1}{1 + e^{-x}}$$

$$a'(x) = a(x)(1 - a(x))$$

Tanh:

$$a(x) = \tanh(x)$$

$$a'(x) = 1 - a(x)^2$$

ReLU:

$$a(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$a'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

Differentiating Loss Functions

Regression tasks

$$L = \frac{1}{2} \sum_l (y_l - \hat{y}_l)^2$$

$$\frac{\partial L}{\partial \hat{y}_i} = \hat{y}_i - y_i$$

Classification tasks

$$L = - \sum_l y_l \log(\hat{p}_l)$$

$\hat{p}_l$ is the predicted probability of class l

Softmax Activation Function

1 2 ... i^{th} neuron

Softmax Activation function

Take $\hat{y}_i$ to be the output of the i^{th} neuron in output layer prior to softmax.

Compute probabilities of each class as

$$\hat{p}_i = \frac{e^{\hat{y}_i}}{\sum_l e^{\hat{y}_l}}$$